

UNIVERSIDADE DE SÃO PAULO — INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO

Padrão de Projeto Observer

Catálogo GoF — Padrão Comportamental

Disciplina: SSC0124 — Análise e Projeto Orientados a Objetos

Aluno: Rafael Feltrim

Docente: Profa. Dra. Lina Garcés

01 Definição do Padrão

O **Observer** é um padrão de projeto comportamental que estabelece uma dependência *um-para-muitos* entre objetos, de forma que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Publicador (Subject)

Objeto que detém o estado principal e gerencia a emissão de notificações sobre suas alterações.

Assinante (Observer)

Objetos dependentes que se registram para receber notificações quando o estado do Publicador for alterado.

02 O Problema

Considere a necessidade de manter consistência entre objetos relacionados sem criar acoplamento forte.

Abordagem 1: Polling Constante

O objeto dependente verifica o estado do objeto principal periodicamente.

Desperdício considerável de recursos computacionais, pois a maioria das verificações não resulta em alterações.

Abordagem 2: Notificação em Massa

O objeto principal notifica indiscriminadamente todos os outros objetos do sistema a cada mudança.

Sobrecarga de comunicação e processamento em objetos que não têm interesse na alteração.

Dilema: Como notificar apenas os objetos interessados, no momento exato da alteração, sem que o emissor precise conhecer a classe concreta dos receptores?

03 A Solução Proposta

O padrão introduz um mecanismo estruturado de inscrição (*Publish-Subscribe*):

1. Inscrição

Objetos *Observers* anexam-se ao *Subject* através de um método público.

2. Alteração de Estado

O *Subject* sofre uma modificação em seu estado interno.

3. Notificação

O *Subject* itera sobre sua lista de inscritos, invocando um método comum de atualização.

Elementos Arquiteturais Fundamentais

- O *Subject* mantém apenas uma lista de referências abstratas (interfaces).
- Métodos de gerenciamento: `attach()` e `detach()`.
- Os *Observers* implementam uma interface comum contendo o método `update()`.

04 Contextualização Abstrata

Analogia: Assinatura de Periódicos

Em vez de um leitor visitar a banca diariamente para verificar se uma nova edição foi lançada (Polling), ele realiza uma assinatura.

A editora mantém um registro de assinantes e, assim que a edição é publicada, a distribui ativamente para os endereços registrados.

O assinante pode cancelar o serviço (unsubscribe) a qualquer momento, removendo-se da lista.

Mapeamento Estrutural

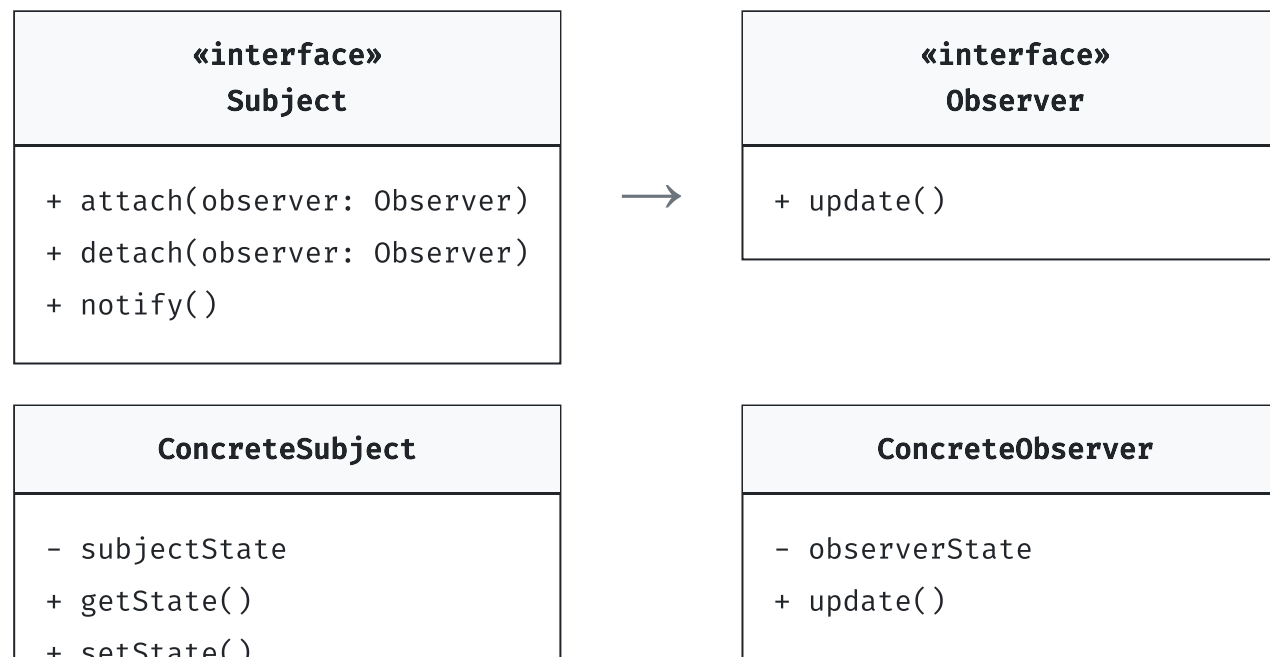
Editora	→	Subject (Publicador)
---------	---	-----------------------------

Lista de Endereços	→	Estrutura de Dados (Coleção)
--------------------	---	-------------------------------------

Leitores Inscritos	→	Observers (Assinantes)
--------------------	---	-------------------------------

Entrega da Edição	→	Método update()
-------------------	---	------------------------

05 Estrutura UML do Padrão



06 Implementação (Java) — Estrutura Base

```
// Interfaces que estabelecem o contrato de comunicação
public interface Subject {
    void attach(Observer observer);
```

```
void detach(Observer observer);
void notifyObservers();
}

public interface Observer {
    void update(float temperature);
}

public class WeatherStation implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperature;

    public void attach(Observer o) { observers.add(o); }
    public void detach(Observer o) { observers.remove(o); }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(temperature);
        }
    }

    public void setTemperature(float t) {
        this.temperature = t;
        notifyObservers(); // Propaga a mudança
    }
}

public class TemperatureDisplay implements Observer {
    private String displayName;
```

07 Aplicabilidade

Cenários Indicados

- Quando a alteração no estado de um objeto exige a modificação de outros, e o conjunto de dependentes é desconhecido ou dinâmico a tempo de execução.
- Quando uma abstração possui dois aspectos onde um depende do outro. Encapsular esses aspectos em objetos separados permite sua variação e reutilização independentes.
- Quando é necessário estabelecer comunicação entre componentes sem introduzir acoplamento rígido (Princípio da Inversão de Dependência).

Contraindicações

- Quando o fluxo de notificação é rigorosamente sequencial (padrões como *Chain of Responsibility* são mais adequados).
- Quando a quantidade de observadores é estática e estritamente limitada, tornando a infraestrutura de eventos desnecessária.

08 Análise Crítica

Vantagens (Prós)

- **Acoplamento Abstrato:** O *Subject* conhece os *Observers* apenas através de uma interface abstrata.
- **Suporte a Broadcast:** A notificação é enviada a todos os objetos inscritos nativamente.
- **Princípio Aberto/Fechado (OCP):** É possível introduzir novas classes de assinantes sem modificar o código do publicador.

Desvantagens (Contras)

- **Atualizações Inesperadas:** Como os observadores não se conhecem, o custo de uma alteração de estado pode ser alto.
- **Ordem não Determinística:** A ordem na qual os observadores são notificados geralmente não é garantida.
- **Vazamento de Memória:** A falha em desregistrar observadores (*Lapsed Listener Problem*) retém objetos na memória.

09 Considerações Finais

Observer vs. Mediator

Enquanto o *Mediator* centraliza o fluxo de comunicação encapsulando as interações em um objeto gestor, o *Observer* distribui a comunicação dinamicamente, conectando emissores e receptores através de interfaces.

Uso Típico em Arquitetura de Software

O Observer é a fundação para arquiteturas reativas (Reactive Programming), arquiteturas orientadas a eventos (Event-Driven Architecture) e o tradicional padrão MVC (Model-View-Controller), no qual a View atua como Observer do Model.

O Padrão Observer demonstra o poder das interfaces em isolar responsabilidades. Ao promover um acoplamento mínimo entre emissores e receptores, tornou-se essencial para a construção de sistemas distribuídos, processamento de eventos e interfaces gráficas responsivas na engenharia de software moderna.

Padrão de Projeto Observer

Padrão Comportamental — Gang of Four (GoF)

Disciplina: SSC0124 — APOO

Aluno: Rafael Feltrim

Data: Agosto/2026

1. Propósito do Padrão

O **Observer** é um padrão de projeto **comportamental** que permite definir um mecanismo de assinatura para notificar múltiplos objetos sobre quaisquer eventos que aconteçam com o objeto que eles estão observando.

O padrão Observer, também conhecido como *Observador*, *Event-Subscriber* ou *Listener*, estabelece uma relação **um-para-muitos** entre objetos, de modo que quando um objeto (o "sujeito" ou "publicador") muda de estado, todos os seus dependentes (os "observadores" ou "assinantes") são automaticamente notificados e atualizados.

Este padrão pertence à categoria dos **padrões comportamentais** do catálogo GoF, sendo um dos 23 padrões originais documentados por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides em 1995.

Aspecto	Detalhes
Nome	Observer (Observador)
Categoria	Comportamental
Também conhecido como	Observador, Event-Subscriber, Listener, Publish-Subscribe
Intenção	Definir uma dependência um-para-muitos entre objetos para que quando um mude de estado, todos os dependentes sejam notificados

2. O Problema que o Padrão Resolve

Imagine um cenário com dois tipos de objetos: um `Cliente` e uma `Loja`. O cliente está interessado em um produto que em breve estará disponível na loja. Há duas abordagens ingênuas para resolver este problema:

2.1. Abordagem por Polling

O cliente visita a loja todos os dias para verificar a disponibilidade do produto. Enquanto o produto não chega, a maioria dessas visitas será em vão, gerando **desperdício de tempo e recursos**.

2.2. Abordagem por Notificação em Massa (Spam)

A loja poderia enviar emails para **todos os clientes** cada vez que um novo produto se torna disponível. Isso economizaria viagens dos clientes interessados, mas **irritaria clientes que não estão interessados** naquele produto específico.

Problema central: Ou o cliente gasta tempo verificando a disponibilidade, ou a loja gasta recursos notificando os clientes errados. Existe um conflito entre eficiência e relevância da comunicação.

Em termos de software, este problema se manifesta quando:

- Um objeto precisa informar outros sobre mudanças em seu estado, mas não sabe quem são (ou quantos são) esses objetos.
- O acoplamento direto entre classes torna o sistema rígido e difícil de estender.
- Novos "observadores" precisam ser adicionados sem modificar o código da classe observada.

3. A Solução Proposta

O padrão Observer propõe um **mecanismo de assinatura** (subscription mechanism) que resolve o problema de forma elegante:

3.1. Conceitos-chave

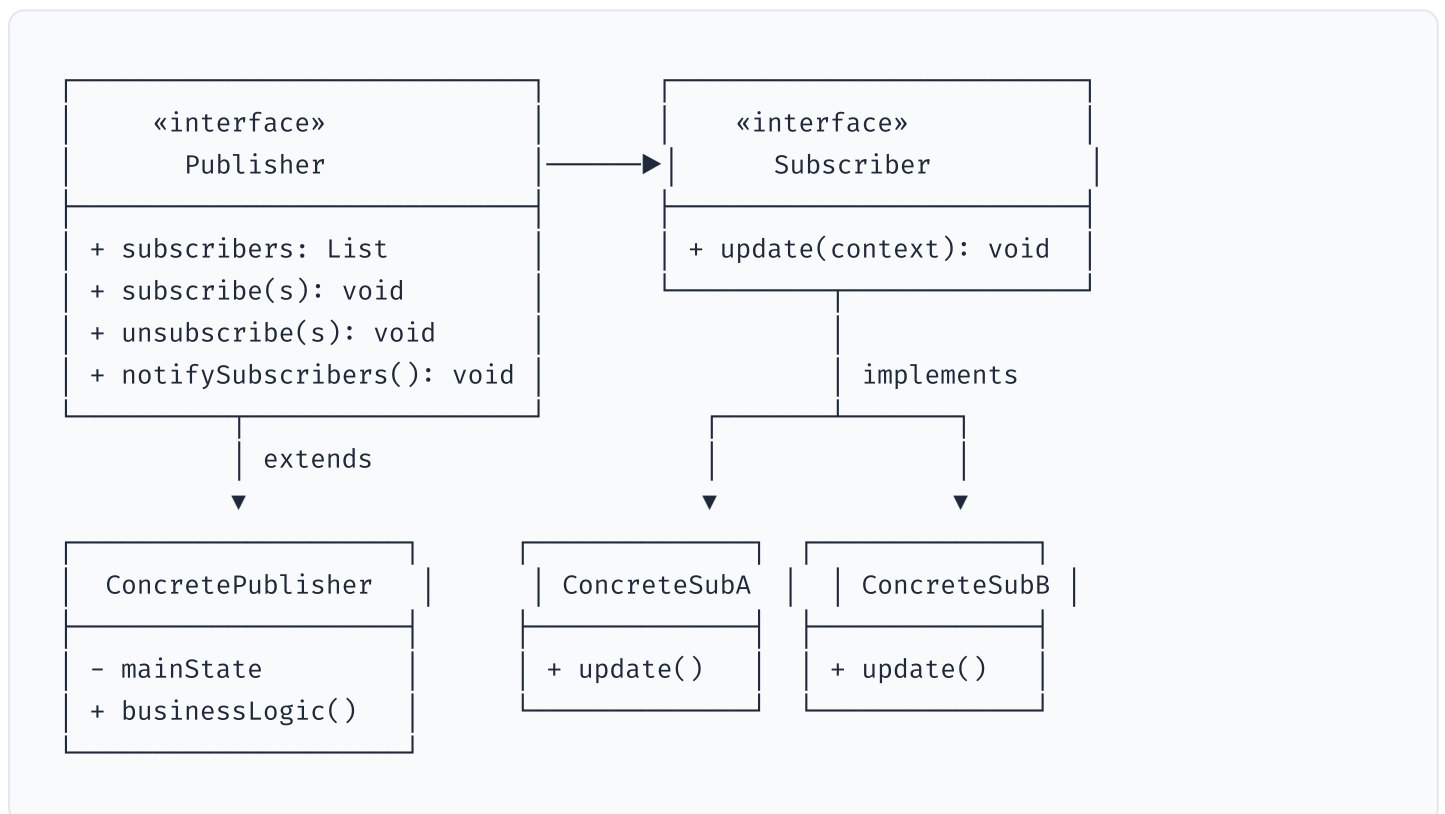
- **Publicador (Publisher/Subject):** O objeto que possui um estado interessante e emite notificações quando esse estado muda.
- **Assinante (Subscriber/Observer):** Os objetos que desejam ser informados sobre mudanças no publicador.
- **Interface do Assinante:** Uma interface comum que todos os assinantes devem implementar, garantindo que o publicador se comunique com eles de forma desacoplada.

3.2. Mecanismo

O mecanismo consiste em:

1. Uma **lista** (vetor) para armazenar referências aos objetos assinantes.
2. Métodos públicos `subscribe()` e `unsubscribe()` para adicionar e remover assinantes.
3. Um método `notify()` que percorre a lista e chama o método `update()` em cada assinante.

3.3. Estrutura (Diagrama de Classes)



3.4. Exemplo em Java — Sensor de Temperatura

Interfaces:

```
public interface Observer {
    void update(float temperatura);
}

public interface Subject {
    void addObserver(Observer o);
    void removeObserver(Observer o);
    void notifyObservers();
}
```

Publicador concreto:

```
public class SensorTemperatura implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperatura;

    public void addObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers)
            o.update(temperatura);
    }

    public void setTemperatura(float t) {
        this.temperatura = t;
        notifyObservers(); // notifica automaticamente
    }
}
```

Assinantes concretos:

```

public class DisplayCelsius implements Observer {
    public void update(float temp) {
        System.out.println("Display: " + temp + "°C");
    }
}

public class AlertaCalor implements Observer {
    public void update(float temp) {
        if (temp > 40)
            System.out.println("⚠️ ALERTA: " + temp + "°C!");
    }
}

```

Uso (Main):

```

public class Main {
    public static void main(String[] args) {
        SensorTemperatura sensor = new SensorTemperatura();

        sensor.addObserver(new DisplayCelsius());
        sensor.addObserver(new AlertaCalor());

        sensor.setTemperatura(25.0f);
        // Saída: Display: 25.0°C

        sensor.setTemperatura(42.0f);
        // Saída: Display: 42.0°C
        //           ⚠️ ALERTA: 42.0°C!
    }
}

```

4. Quando Aplicar o Padrão

4.1. Cenários de Aplicação

- **Mudanças em cascata:** Quando mudanças no estado de um objeto precisam atualizar outros objetos, e o conjunto de objetos é desconhecido de antemão ou muda dinamicamente.

- **Interfaces gráficas:** Botões, campos de texto e outros componentes de GUI que precisam notificar listeners quando eventos ocorrem (cliques, digitação, etc.).
- **Observação temporária:** Quando objetos devem observar outros apenas por um tempo limitado ou em casos específicos — a lista de inscrição é dinâmica.
- **Arquiteturas dirigidas a eventos:** Sistemas de mensageria, event buses, e arquiteturas pub/sub.
- **Desacoplamento:** Quando você quer que o emissor de eventos não conheça as classes concretas que reagem a eles.

4.2. Exemplos no Mundo Real

Tecnologia	Uso do Observer
Java Swing/JavaFX	<code>ActionListener</code> , <code>MouseListener</code> , <code>EventListener</code>
JavaScript/DOM	<code>addEventListener()</code> , <code>EventTarget</code>
Node.js	<code>EventEmitter</code>
Android	<code>LiveData</code> + <code>Observer</code> , <code>Flow</code> , <code>BroadcastReceiver</code>
React/Frontend	Redux, Context API, <code>useEffect</code> , Zustand
Arquitetura	Apache Kafka, RabbitMQ, AWS SNS (Pub/Sub)

5. Prós e Contras

👍 Prós

- **Princípio Aberto/Fechado (OCP):**
Novas classes assinantes podem ser introduzidas sem modificar o código da publicadora, e vice-versa.
- **Relações dinâmicas em runtime:**
Assinantes podem entrar e sair da lista a qualquer momento durante a execução do programa.
- **Baixo acoplamento:** O publicador conhece os assinantes apenas pela interface, não pelas classes concretas. Isso promove a extensibilidade.
- **Reutilização:** O mecanismo de eventos pode ser extraído para uma classe auxiliar e reutilizado em qualquer publicador.

👎 Contras

- **Ordem não determinística:**
Assinantes são notificados em ordem aleatória (ou na ordem de registro), sem garantia de sequência.
- **Risco de memory leaks:** Se assinantes não forem explicitamente removidos, o publicador mantém referências que impedem o garbage collector.
- **Complexidade de depuração:** O fluxo de controle pode ser difícil de rastrear quando há muitos assinantes e eventos encadeados.
- **Atualizações em cascata:** Uma notificação pode desencadear outras notificações, criando potenciais ciclos ou comportamentos inesperados.

6. Relações com Outros Padrões

Padrão Relacionado	Relação
Mediator	O Mediator elimina dependências múltiplas via um objeto central. O Observer estabelece comunicação de uma via dinâmica. Um Mediator pode ser implementado usando Observer internamente.
Command	O Command encapsula uma requisição como objeto. Pode ser combinado com Observer: um comando é executado em resposta a uma notificação.

Chain of Responsibility	Ambos conectam remetentes e destinatários. No CoR, o pedido segue uma cadeia sequencial. No Observer, a notificação vai para todos os inscritos simultaneamente.
Strategy	Ambos usam composição e interfaces. Strategy permite trocar algoritmos; Observer permite reagir a eventos. São complementares.

7. Conexão com Princípios SOLID

- **S — Single Responsibility:** O publicador se preocupa com sua lógica de negócio; a gestão de assinantes pode ser delegada a uma classe auxiliar (ex: `EventManager`).
- **O — Open/Closed:** O principal benefício. Novos assinantes são adicionados sem modificar o publicador.
- **D — Dependency Inversion:** O publicador depende da abstração (interface `Subscriber`), não de implementações concretas.

Referências

1. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
2. Valente, M. T. *Engenharia de Software Moderna*. Cap. 6: Padrões de Projeto. Disponível em: <https://engsoftmoderna.info/cap6.html>
3. Refactoring.Guru. *Observer*. Disponível em: <https://refactoring.guru/pt-br/design-patterns/observer>
4. Refactoring.Guru. *O que é um padrão de projeto?* Disponível em: <https://refactoring.guru/pt-br/design-patterns/what-is-pattern>
5. Refactoring.Guru. *Classificação dos padrões*. Disponível em: <https://refactoring.guru/pt-br/design-patterns/classification>